

Module 21

Pivoting & Lateral Movement

Routing through a controlled foothold: multi-homed hosts, SSH forwarding, SOCKS, ProxyJump, ProxyChains, route-based tunnels, segmentation, lateral-movement reasoning, telemetry, dan cleanup.

Learning path module ini

Network visibility -> multi-homed foothold -> local/dynamic/remote forwarding -> SOCKS -> internal enumeration -> ProxyJump lateral movement -> Ligolo-ng/Chisel mental model -> telemetry -> cleanup -> capstone.

Exit criteria

- Bisa membaca interface dan routing table lalu menjelaskan kenapa satu host dapat menjadi pivot tanpa mengaktifkan IP forwarding.
- Bisa membedakan SSH local (-L), dynamic (-D), remote (-R), dan ProxyJump (-J), termasuk arah listener dan connection origin.
- Bisa memakai SOCKS + ProxyChains secara terbatas pada TCP dan memahami kenapa beberapa scanner/protocol tidak cocok diproxy-kan.
- Bisa membedakan pivoting (network path) dari lateral movement (berpindah execution/authentication context ke host lain).
- Bisa memilih antara port-forward, SOCKS, TUN/routed tunnel, atau jump host berdasarkan kebutuhan, scope, dan blast radius.

Authorized-lab rule Hands-on module ini hanya untuk Docker lab lokal yang disediakan, GOAD/HTB/THM yang authorized, atau jaringan milik sendiri. Jangan membuat tunnel ke jaringan perusahaan/kampus/pihak lain tanpa Rules of Engagement yang eksplisit.

1. Mental Model: Reachability Bukan Ownership

Pivoting memperluas jalur network dari posisi yang sudah kita miliki. Lateral movement adalah penggunaan identity/session/remote-access path untuk memperoleh execution context pada host lain. Keduanya sering berurutan tetapi bukan hal yang sama.

Concept	Pertanyaan inti	Contoh
Segmentation	siapa bisa berbicara langsung ke siapa?	attacker tidak punya route ke subnet internal
Pivot	host mana punya dua visibility zone?	jump host punya interface external + internal
Tunnel	bagaimana traffic melewati pivot?	SSH -L/-D/-R, Chisel, Ligolo-ng
Lateral movement	identity/session pindah ke host mana?	SSH via ProxyJump ke internal host
Objective	akses minimum apa yang diperlukan?	reach internal web, bukan scan seluruh subnet

ATTACKER (172.31.210.10)

| external only

v

JUMP (172.31.210.20 + 172.31.211.20)

| internal visibility

+-----> WEB 172.31.211.30:80

+-----> SSH 172.31.211.40:22

Lab 01 - Workspace & topology

./labs/workspace-init.sh module21-pivoting

python ./labs/topology-report.py

2. Routing, Interfaces, dan Multi-Homed Hosts

Sebelum membuat tunnel, identifikasi interface, route, dan reachable networks. Host multi-homed dapat mengakses lebih dari satu segment, tetapi itu tidak otomatis berarti kernel melakukan forwarding antar-segment. SSH forwarding bekerja karena proses sshd pada jump host membuka koneksi baru ke destination dari network namespace miliknya.

Command	Jawaban yang dicari
ip -brief addr	interface + address apa yang ada?
ip route	network mana yang directly connected / lewat gateway?
ip route get <IP>	kernel memilih path/interface mana?
ss -lnp	listener apa yang aktif dan di address mana?
curl / nc	apakah protocol benar-benar reachable?

Omarchy tooling

omarchy pkg add openssh proxychains-ng nmap curl netcat-openbsd

iproute2 biasanya sudah bagian dari base system; cek: ip -V

Lab 02 - Build isolated pivot lab

```
cd modules/21-pivoting-lateral-movement/labs
./pivot-lab.sh build
./pivot-lab.sh create
./pivot-lab.sh status
./segmentation-check.sh
```

Expected result Dari attacker container, 172.31.210.20:22 reachable. 172.31.211.30 tidak dapat diakses langsung. Ini membuktikan segmentation sebelum kita membuat tunnel.

3. SSH Local Port Forward (-L)

Local forwarding membuat listener di sisi client. Setiap koneksi ke listener lokal diteruskan melalui SSH channel, lalu sshd di jump host membuat koneksi ke destination yang kita tentukan.

```
local client      jump      internal
127.0.0.1:8088 --SSH channel--> 172.31.210.20 --TCP connect--> 172.31.211.30:80
```

Lab 03 - Forward satu internal web service

```
# terminal A: masuk attacker container
./pivot-lab.sh shell
```

```
ssh -N \
-o ExitOnForwardFailure=yes \
-L 127.0.0.1:8088:172.31.211.30:80 \
pivot@172.31.210.20
# password training: Pivot21!
# terminal B: masuk attacker container lagi
curl -i http://127.0.0.1:8088/
```

- Bind ke 127.0.0.1 supaya listener tidak terekspos ke container/network lain.
- Gunakan -N jika tunnel tidak membutuhkan remote shell.
- ExitOnForwardFailure=yes membuat command gagal jika listener tidak bisa dibangun, bukan diam-diam lanjut.

Think in direction -L cocok saat operator ingin mengekspos satu destination internal sebagai port lokal. Ini paling kecil blast radius-nya dibanding membuat seluruh subnet routable.

4. Dynamic Forwarding (-D) + SOCKS + ProxyChains

Dynamic forwarding membuat SSH client bertindak sebagai SOCKS4/5 server. Aplikasi yang memahami SOCKS dapat meminta koneksi ke banyak destination melalui satu SSH channel. ProxyChains mencoba memaksa program TCP yang tidak punya native proxy support melewati SOCKS/HTTP proxy.

Lab 04 - Buat SOCKS listener

```
# terminal A di attacker container
ssh -N \
-o ExitOnForwardFailure=yes \
-D 127.0.0.1:1080 \
pivot@172.31.210.20
```

Lab 05 - Akses internal web via ProxyChains

```
# terminal B di attacker container
proxychains4 -f /etc/proxychains4.conf \
curl -s http://172.31.211.30/
```

Lab 06 - Internal enumeration yang compatible dengan SOCKS

```
proxychains4 -f /etc/proxychains4.conf \
nmap -sT -Pn -n -p 22,80 \
172.31.211.30 172.31.211.40
```

ProxyChains limitation ProxyChains-NG meredirect koneksi TCP dari program dynamically linked. Ia tidak membawa ICMP/UDP dan pcap/raw-packet scan tidak cocok. Untuk Nmap gunakan TCP connect (-sT), numeric IP, port terbatas, dan validasi manual.

5. ProxyJump (-J) dan Lateral Movement

ProxyJump meminta OpenSSH membuat koneksi ke destination melalui jump host. Ini bukan "scan through proxy"; ini transport SSH ke host akhir. Dalam lab, credential target sudah disediakan dan tidak diperoleh lewat spraying atau dumping.

Lab 07 - SSH ke internal host melalui jump

```
# dari attacker container
ssh -J pivot@172.31.210.20 \
student@172.31.211.40
```

```
# passwords training:
```

```
# pivot -> Pivot21!
# student -> Student21!
# setelah login ke internal host
hostname
id
ip -brief addr
ip route
ss -lnt
```

Lateral-movement boundary Di titik ini terjadi perubahan execution context: command sekarang berjalan pada internal-ssh sebagai user student. Itu berbeda dari sekadar mengirim traffic melalui jump.

Safer SSH config pattern

```
Host rt21-jump
  HostName 172.31.210.20
  User pivot
```

```
Host rt21-internal
  HostName 172.31.211.40
  User student
  ProxyJump rt21-jump
```

6. SSH Remote Forward (-R): Membalik Arah Listener

Remote forwarding membuat listener pada sisi SSH server. Koneksi yang masuk ke listener remote dibawa melalui SSH channel dan diteruskan ke destination di sisi client. Ini sering membingungkan karena arah listener berlawanan dengan -L.

Lab 08 - Remote listener hanya di localhost jump

```
# terminal A di attacker container
python3 -m http.server 9000 --bind 127.0.0.1
```

```
# terminal B di attacker container
ssh -N \
  -o ExitOnForwardFailure=yes \
  -R 127.0.0.1:9001:127.0.0.1:9000 \
  pivot@172.31.210.20
```

```
# terminal C di Omarchy host
./pivot-lab.sh jump-shell
curl -I http://127.0.0.1:9001/
```

Security detail Bind address eksplisit + GatewayPorts=no mencegah remote-forward listener tersedia ke seluruh network jump. Dalam engagement nyata, listener exposure harus selalu masuk design review.

7. Choosing the Tunnel Primitive

Need	Good starting primitive	Trade-off
1 internal TCP service	SSH -L	simple, minimal exposure, one mapping per service
many TCP destinations	SSH -D + SOCKS	flexible; app/protocol compatibility matters
SSH login through bastion	ProxyJump	clean native SSH workflow; SSH-specific
remote-side listener back to operator	SSH -R	direction is easy to misconfigure; bind carefully
route-like access to subnet	Ligolo-ng TUN	natural routing + TCP/UDP/ICMP; TUN/route changes on proxy
HTTP-carried forward/reverse tunnels	Chisel	single binary, SSH-secured-over-HTTP; still process + long-lived channel

Lab 09 - Record why, not just how

```
python ./labs/activity-log.py module21-pivoting/06-evidence/activity.tsv \
  --action 'dynamic-socks' \
  --source attacker \
  --destination '172.31.211.0/24 via jump' \
  --purpose 'enumerate only TCP 22/80 in training lab'
```

8. Ligolo-ng: Route-Based Pivoting Mental Model

Ligolo-ng memakai TUN interface pada proxy/operator side dan userland network stack pada agent side. Hasilnya, operator menambahkan route ke subnet agent dan banyak tool dapat memakai normal socket/routing tanpa ProxyChains. Agent side tidak selalu membutuhkan privilege tinggi; proxy side perlu kemampuan membuat TUN interface.

```
Operator route table
172.31.211.0/24 -> ligolo TUN
|
v
Ligolo proxy === encrypted connection === Ligolo agent
|
+--> internal network
```

Lab 10 - Route planning (tanpa deploy agent)

- Dari topology.json, identifikasi network yang akan ditambahkan sebagai route: 172.31.211.0/24.
- Tulis evidence yang harus diambil sebelum route ditambah: ip route, interface list, scope, dan expected next hop.
- Tulis cleanup: stop tunnel, hapus route/interface yang dibuat, verifikasi route table kembali ke baseline.

Certificate hygiene Dokumentasi Ligolo-ng mendukung self-signed TLS + fingerprint verification. Opsi yang mengabaikan certificate verification sebaiknya hanya dipakai debugging lab; jangan jadikan default operasi.

9. Chisel: HTTP Transport + SSH Security

Chisel adalah single-binary TCP/UDP tunnel yang dibawa lewat HTTP dan diamankan dengan SSH. Ia mendukung normal forward, reverse forward, SOCKS, authentication, server fingerprint, dan beberapa tunnel dalam satu connection. Ini useful ketika SSH forwarding native tidak tersedia tetapi HTTP connectivity ada.

Lab 11 - Tool provenance & help-only inspection

optional; tidak diperlukan untuk core lab
 docker run --rm jpillora/chisel --help

alternatif: build dari source pada pinned/reviewed version
 # daripada menjalankan curl | shell di workstation security-sensitive

- Sebelum deploy: catat version/source, authentication mode, fingerprint/TLS model, bind addresses, allowed remotes, dan cleanup plan.
- Reverse forwarding memperluas listener surface; jangan gunakan wildcard bind jika localhost sudah cukup.
- Perubahan upstream dapat memengaruhi ACL/SOCKS behavior; baca README/release notes versi yang benar-benar digunakan.

10. Multi-Hop Pivot Chains: Complexity Naik Cepat

Satu jump host masih mudah dipahami. Dua atau tiga pivot membuat routing, DNS, credential scope, failure domain, dan cleanup jauh lebih kompleks. Jangan membuat chain hanya karena tool mendukungnya; pilih chain minimum yang dibutuhkan untuk objective.

Operator -> Jump A -> Jump B -> Internal Service

Setiap hop menambah:
 + authentication event
 + long-lived connection
 + failure point
 + listener/route state
 + cleanup obligation

Lab 12 - Multi-hop design tabletop

- Tanpa menambah container baru, gambar hypothetical second jump di 172.31.211.50 yang punya akses ke 172.31.212.0/24.
- Tentukan apakah objective hanya satu TCP service (pilih chained local forward/ProxyJump) atau butuh route-like access (pertimbangkan TUN-based tunnel).
- Tuliskan failure behavior jika Jump A restart, Jump B kehilangan route, atau satu credential dicabut.

Operator rule Setiap hop harus punya owner, purpose, start time, expected end time, bind/route state, dan cleanup record. Chain tanpa inventory hampir selalu menghasilkan orphan tunnel.

11. DNS Through a Pivot: Sering Jadi Sumber False Negative

Internal hostname sering hanya dikenal oleh resolver internal. Jika aplikasi resolve nama sebelum traffic masuk tunnel, query dapat bocor ke resolver operator atau gagal total. Tentukan apakah resolution terjadi lokal, melalui SOCKS remote DNS, atau dari routed network setelah TUN aktif.

Pattern	Resolution location	Risk / note
Numeric IP	none	paling deterministik untuk lab/known assets
Local DNS	operator resolver	internal zone mungkin tidak dikenal; dapat leak query
ProxyChains proxy_dns	melalui proxy mechanism	hanya cocok untuk tool yang benar-benar ter-hook
Route-based + internal DNS	operator mengakses resolver via route	lebih natural, tetapi scope resolver juga harus authorized

Lab 13 - DNS decision record

- Untuk core Docker lab gunakan numeric IP sehingga hasil tidak bergantung DNS.
- Tuliskan bagaimana strategi berubah jika target diberi nama app.internal.example dan hanya resolver 172.31.211.53 yang mengetahuinya.

- Catat DNS sebagai bagian dari tunnel plan, bukan troubleshooting terakhir.

12. Lateral Movement: Transport, Identity, dan Execution Context

Lateral movement harus dibaca sebagai kombinasi transport + authentication material + authorization + remote execution/session primitive. Module ini hanya mempraktikkan SSH dengan training credentials; protocol lain dibahas sebagai decision model agar tidak tercampur dengan credential abuse.

Method	Needs	Execution/session result	Telemetry examples
SSH	network + valid account/key	shell/command session	sshd auth, process tree, source IP
RDP	network + interactive-logon right	desktop session	4624/logon type, RDP operational logs
WinRM	network + authorized account/policy	PowerShell/remoting session	WinRM + PowerShell logs
SMB/RPC remote admin	network + appropriate admin rights	service/task/RPC dependent	SMB/RPC + service/task events

No credential guessing here Password spraying, hash replay, ticket abuse, credential dumping, dan remote-service abuse tidak dilakukan di module ini. Jika identity belum authorized untuk target, berhenti pada path hypothesis.

Lab 14 - Lateral evidence record

```
python ./labs/activity-log.py module21-pivoting/06-evidence/activity.tsv --action 'proxyjump-login' --source 'attacker via jump' --destination '172.31.211.40:ssh' --purpose 'authorized student shell for topology validation'
```

13. Tunnel Security & Credential Hygiene

- Jangan aktifkan SSH agent forwarding (-A) secara default. Host remote yang dapat mengakses forwarded agent socket dapat memakai agent untuk autentikasi selama koneksi hidup.
- Bind listener ke 127.0.0.1 kecuali Rules of Engagement memang membutuhkan listener di interface lain.
- Gunakan host-key/fingerprint verification. Convenience option yang mematikan certificate/host verification hanya untuk debugging disposable lab.
- Jangan menaruh password/token di command line, process list, shell history, atau README repo. Training password di module ini sengaja disposable.
- Gunakan ServerAliveInterval/ServerAliveCountMax untuk menghindari tunnel zombie, dan ExitOnForwardFailure untuk fail-fast.

```
ssh -N -o ExitOnForwardFailure=yes -o ServerAliveInterval=30 -o ServerAliveCountMax=3 -L 127.0.0.1:8088:172.31.211.30:80 pivot@172.31.210.20
```

Lab 15 - Inspect tunnel state

```
# attacker container
ss -lntp | grep -E ':(8088|1080|9000)' || true
ps aux | grep '[s]sh.*172.31.210.20'
```

```
# jump container from Omarchy host
./labs/pivot-lab.sh jump-shell
ss -tnp
ps aux | grep '[s]shd'
```

14. Segmentation, DNS, dan Tool Behavior Setelah Pivot

Setelah tunnel bekerja, jangan langsung melakukan full-subnet scan. Bangun inventory bertahap: route -> known hosts -> known ports -> protocol verification. DNS juga harus dipikirkan: resolver lokal operator mungkin tidak mengenal internal names, sementara proxy-side resolver atau SOCKS remote DNS dapat menghasilkan hasil berbeda.

Problem	Symptom	Response
No route	network unreachable / timeout	verify ip route + intended tunnel state
SOCKS works, ping fails	expected: ICMP not proxied	use TCP validation or route-based tunnel
Name fails but IP works	DNS not traversing intended path	use numeric IP or controlled remote DNS
Nmap strange through SOCKS	raw/pcap method incompatible	use -sT, small port set, manual verify
Forward listener unreachable	wrong bind or collision	ss -lntp + explicit 127.0.0.1 bind

Lab 16 - Evidence & cleanup

```
# capture before cleanup
ip route > module21-pivoting/06-evidence/attacker-routes.txt
ss -lntp > module21-pivoting/06-evidence/listeners.txt
./labs/evidence-manifest.sh
```

```
# stop SSH forward processes, exit lab shells, then:
./labs/pivot-lab.sh cleanup
```

15. Detection-Aware View

Pivoting menghasilkan telemetry meskipun payload tidak dieksekusi pada internal target. Defender dapat melihat long-lived SSH/TLS/HTTP connections, new listeners, process execution, authentication, unusual east-west connections, serta a workstation yang tiba-tiba menjadi connection broker.

Activity	Potential telemetry
SSH -L/-D/-R	sshd auth/session logs, long-lived connection, local/remote listener, child connections
ProxyJump lateral login	authentication logs pada jump + target, new user session/processes
ProxyChains enumeration	many TCP connects sourced from pivot/jump context
Ligolo-ng / Chisel	new binary/process, persistent encrypted egress, route/TUN/listener changes
Cleanup failure	orphan listener/process/route remains after engagement

Blue Team perspective Network segmentation bukan hanya firewall rule. Monitor siapa yang menjadi unexpected bridge antara zones: dual-homed hosts, admin workstations, jump servers, VPN endpoints, CI runners, dan management interfaces.

16. Capstone - Reach the Internal Zone with Minimum Exposure

Scenario: attacker hanya berada di external Docker network. Objective adalah membuktikan internal web reachable dan melakukan satu authorized SSH login ke internal host melalui jump, dengan exposure minimum dan evidence yang cukup untuk direproduce.

- Phase A - Baseline: buktikan direct access ke 172.31.211.30 gagal dari attacker.
- Phase B - Local forward: expose hanya internal web sebagai 127.0.0.1:8088 dan capture HTTP evidence.
- Phase C - Dynamic SOCKS: enumerate hanya TCP 22 dan 80 pada dua known internal hosts; jangan scan /24 penuh.
- Phase D - Lateral movement: gunakan provided training credentials + ProxyJump untuk login ke 172.31.211.40 sebagai student.
- Phase E - Evidence: simpan route/listener/output, timestamp activity, hash evidence.
- Phase F - Cleanup: stop tunnels, exit sessions, remove containers/networks, verifikasi tidak ada listener training tersisa.

Deliverable

Artifact	Minimum content
topology.md	zones, node IPs, why jump is pivot candidate
tunnel-plan.md	chosen primitive + bind + destination + reason
activity.tsv	timestamped actions and intended scope
evidence/	curl/nmap/ssh observations + route/listener snapshots
cleanup.md	process/listener/network removed + validation commands

17. Cheat Sheet & Common Mistakes

Need	Pattern
Local forward	ssh -N -L 127.0.0.1:LPORT:DEST:DPORT user@jump
Dynamic SOCKS	ssh -N -D 127.0.0.1:1080 user@jump
Remote forward	ssh -N -R 127.0.0.1:RPORT:LOCAL:LPORT user@jump
SSH bastion	ssh -J user@jump user@internal
Route check	ip -brief addr ; ip route ; ip route get DEST
Listener check	ss -lntp
SOCKS TCP scan	proxychains4 nmap -sT -Pn -n -p <small-set> <known-host>

Common mistake Mengikat listener ke 0.0.0.0 ketika 127.0.0.1 cukup, menjalankan full subnet scan begitu tunnel hidup, menganggap SOCKS membawa semua protocol, lupa DNS path, atau meninggalkan tunnel/process setelah cleanup.

Knowledge Check

Question A	Question B
☐ Kenapa dual-homed host tidak otomatis L3-forward?	☐ Listener -L, -D, dan -R berada di sisi mana?
☐ Kenapa ProxyChains + Nmap pilih -sT?	☐ Pivoting beda apa dengan lateral movement?
☐ Kapan single forward lebih baik dari routed tunnel?	☐ Kenapa bind address + DNS path masuk threat model?
☐ Telemetry apa muncul saat host jadi pivot?	☐ Bukti minimum cleanup tunnel berhasil apa saja?

References & Further Reading

Source	URL
OpenSSH ssh(1) - forwarding + ProxyJump	https://man.openbsd.org/ssh.1
Linux ssh(1) - forwarding semantics	https://man7.org/linux/man-pages/man1/ssh.1.html
ip-route(8) - routing table management	https://man7.org/linux/man-pages/man8/ip-route.8.html
ProxyChains-NG - SOCKS/TCP limitations	https://github.com/rofl0r/proxychains-ng
Arch proxychains-ng / OpenSSH	https://archlinux.org/packages/extra/x86_64/proxychains-ng/
Ligolo-ng docs - TUN/routing model	https://docs.ligolo.ng/
Chisel official repository	https://github.com/jpillora/chisel
Docker networking overview	https://docs.docker.com/engine/network/